

Chapter 1: Generation of the Static Parent Data — Self-Consistent Circularly Polarized Eigenmode Parents and Zero-Closure Kernel Seeds for N=3..40

Noriaki Kihara (WF System Co., Ltd.) ORCID 0009-0004-6753-4020

September 5, 2026 Version DOI 10.5281/zenodo.22317636

(Chapter 1 of the reproduction papers for the N=3..40 stage-1+2+3 sweep. Shares the Concept DOI 10.5281/zenodo.22317635 with the overview paper; Version DOI 10.5281/zenodo.22317636. Code blocks are quoted verbatim from the programs; comments inside them remain in Japanese, as in the originals.)

1. Purpose

This chapter describes the numerical experiment that deterministically generates, from a random-seed formula, the initial data used by the sweep of Chapter 2 — the self-consistent circularly polarized eigenmode parent v , the zero-closure kernel seed g , and the normalized initial state $Z0$ — and fixes them as static files (npz).

The generation procedure, the order of random-number consumption, and all arguments are identical to the canonical run of 2026-07-22 (自発の分裂予備実験_v1/run_spontaneous_splitting_largeN_v1.py; N=40, $\delta=1e-15$, seed=0, tol=1e-12), and the pass gate is that, for N=40, the generated objects are bit-identical to the initial values of the canonical run. The purpose of this chapter is complete reproducibility and the fixing of the correspondence among equations, programs, and data; it contains no physical interpretation.

2. Theoretical Background

We work with complex relation waves on the edge set of the complete graph K_N .

(Eq. 1) Edge set and dimension — The number of edges is $M = N(N - 1)/2$. The edge ordering is fixed to the upper-triangular order $((0,1),(0,2),\dots,(0,N - 1),(1,2),\dots)$. The state is $z \in \mathbb{C}^M$.

(Eq. 2) Phase-difference sine generator — For edge phases $\theta \in \mathbb{R}^M$, on vertex-sharing edge pairs (e,f) :

$$K_{ef}(\theta) = \cos \theta_e \sin \theta_f - \sin \theta_e \cos \theta_f = \sin(\theta_f - \theta_e)$$

0 on non-adjacent pairs, 0 on the diagonal. K is real antisymmetric ($K^T = -K$).

(Eq. 3) Vertex decomposition and the small space — Scattering $c_e = \cos \theta_e$, $s_e = \sin \theta_e$ to the vertices yields matrices C, S ($M \times N$); with $W = [C|S]$ ($M \times 2N$) and $J = [[0, I_N], [-I_N, 0]]$:

$$K = C S^T - S C^T = W J W^T, \quad \text{rank } K \leq 2N$$

The Gram matrix $G = W^T W$ ($2N \times 2N$) can be constructed analytically because, in the line graph of K_N , a vertex pair (k,l) shares exactly one edge (the edge (k,l)):

$$G_{cc}[k,l] = \cos^2 \theta_{\{(k,l)\}}, \quad G_{cs}[k,l] = \cos \theta_{\{(k,l)\}} \sin \theta_{\{(k,l)\}}, \quad G_{ss}[k,l] = \sin^2 \theta_{\{(k,l)\}} \quad (k \neq l)$$

diagonals are the row sums of the off-diagonals

(Eq. 4) Circularly polarized eigenmode (parent) — The nonzero spectrum of K corresponds to the eigenvalues of JG ($2N \times 2N$). From the eigenvector y of the eigenvalue λ with the smallest imaginary part ($\lambda = -i \sigma_{\max}$):

$$v = W y, \quad v \leftarrow v / \|v\|$$

Since iK is Hermitian, if v is an eigenmode then $iKv = \mu v$ ($\mu = \sigma_{\max}$).

(Eq. 5) Self-consistent iteration (phase mixing) — The phases $\theta_{\text{new}} = \arg v$ read from v are mixed into the current phases with mixing ratio $\beta = 0.5$:

$$\theta \leftarrow \arg((1 - \beta) e^{i\theta} + \beta e^{i\theta_{\text{new}}})$$

A fixed point of this iteration is the self-consistent parent: “the eigenmode of the generator built from its own phases is itself”.

(Eq. 6) Eigenmode residual (convergence criterion) —

$$\mu = \text{Re}(v^\dagger (iKv)), \quad r = \|iKv - \mu v\|$$

Convergence is declared when $r < \text{tol}$ ($\text{tol} = 1e-12$ in this chapter).

(Eq. 7) Zero-closure kernel seed — With the projection onto the orthogonal complement of $\text{span}(W)$

$$P \perp g = g - W q, \quad q = \text{lstsq}(G, W^T g)$$

construct from real Gaussian vectors ξ_1, ξ_2 : $u = P \perp \xi_1 / \|\cdot\|$ and $w =$ the orthonormalized $P \perp \xi_2$, and set

$$g = (u + i w) / \sqrt{2}$$

From $u \perp w$ and $\|u\| = \|w\| = 1$, $g^T T g = (\|u\|^2 - \|w\|^2)/2 + i u \cdot w = 0$ holds exactly (zero closure). Moreover $g \in \text{span}(W)^\perp$, i.e., g lies outside the range of K .

(Eq. 8) Initial state —

$$z_0 = (v + \delta g) / \|v + \delta g\|, \quad \delta = 1e-15$$

(Eq. 9) Random numbers — The RNG is numpy PCG64 with seed

$$\text{seed}(N) = 40260722 + 1000 \cdot N + 0$$

The consumption order is fixed: “initial phases θ^0 of the parent iteration (m uniform numbers, per restart) $\rightarrow \xi_1$ (m normal numbers) $\rightarrow \xi_2$ (m normal numbers)”. All generated objects are thereby deterministic functions of N alone.

3. Implementation Method

- The engine `run_n_scaling_lowrank_v1.py` is a **bit-identical copy** of the 2026-07-22 canonical (自発の分裂予備実験_v1/run_n_scaling_lowrank_v1.py), verified by diff. The implementations of Eqs. 2–7 all reside in this engine; this chapter uses them **unmodified, via import only** (following the rule prohibiting independent reimplementation).
- The generator program `make_static_parents_N3_N40_v1.py` executes, for $N=3..40$, the same call sequence as the opening of the canonical `run()` (rng of Eq. 9 $\rightarrow$ `make_parent` $\rightarrow$ `zero_closure_kernel_seed` $\rightarrow$ normalization of Eq. 8) and saves to npz.
- For $N=40$, bit-identity with the canonical static parent (2026-09-04; itself verified bit-identical against the canonical run) is checked as an in-program gate.

4. Detailed Design

4.1 Overall Flow

for N in $3..40$:

[initialization] build LowRankSystem(N) (edge order, J), `rng = default_rng(40260722+1000N)`

[loop] `make_parent`: self-consistent iteration (Eq. 4 $\rightarrow$ Eq. 5 $\rightarrow$ Eq. 6; up to 1200 iterations $\times$ 3
 `zero_closure_kernel_seed`: construction of the seed g (Eq. 7)

[finalization] build Z_0 (Eq. 8) $\rightarrow$ $N=40$ gate $\rightarrow$ save npz (verify only if file exists)

$\rightarrow$ append to the ledger

write the ledger `parents_summary.csv`; exit 1 if any gate fails

4.2 Overall Data Flow

- **Input:** no file input. Only the random-seed formula (Eq. 9) and constants.
 - `SEED` = 0 ... third term of the seed formula (series number)
 - `DELTA` = $1e-15$... δ of Eq. 8 (seed amplitude)
 - `TOL` = $1e-12$... convergence threshold of Eq. 6 (identical to the canonical run's `--tol=1e-12`)
 - `ITERS` = 1200 ... iteration cap of the self-consistent loop (identical to the canonical `run()`'s `iters=1200`)
 - reference input `REF40` ... the canonical static-parent npz for the $N=40$ gate (read-only)
- **Processing:** repeat the individual processes of 4.3 for $N=3..40$.
- **Output:**
 - `parents/parent_static_N{N:05d}_makeparent_20260905.npz` $\times$ 38 (fields: `v`, `g`, `Z0`, `sigma` (the parent's σ spectrum), `residual`, `n`, `seed`, `delta`, `tol`, `iters`)
 - `parents/parents_summary.csv` (columns: `N`, `M`, `parent_residual`, `rank_planes`, `status`)

Constant definitions (`make_static_parents_N3_N40_v1.py`):

```
34 PARENT_DIR = os.path.join(BASE_DIR, "parents")
35 REF40 = '.../自発の分裂予備実験_v1_N40 对照実験系_20260904/largeN_splitting_result_v1/parent_sta
36
37 SEED = 0
```

```

38 DELTA = 1e-15
39 TOL = 1e-12
40 ITERS = 1200

```

4.3 Individual Processes

■4.3.1 Initialization Engine initialization (Eq. 1; J of Eq. 3). The edge order is fixed by `np.triu_indices`.

`run_n_scaling_lowrank_v1.py`:

```

52 def build_edges(n):
53     ea, eb = np.triu_indices(n, k=1)
54     return ea.astype(np.int64), eb.astype(np.int64)
...
60 def __init__(self, n):
61     self.n = n
62     self.ea, self.eb = build_edges(n)
63     self.m = len(self.ea)
64     self.J = np.zeros((2 * n, 2 * n))
65     self.J[:n, n:] = np.eye(n)
66     self.J[n:, :n] = -np.eye(n)

```

Caller side (`make_static_parents_N3_N40_v1.py`, Eq. 9):

```

48 sys_lr = LowRankSystem(N)
49 rng = np.random.default_rng(40260722 + 1000 * N + SEED)

```

■4.3.2 Loop (1): Construction of the Self-Consistent Parent (Eqs. 2–6) Generator construction (Eqs. 2, 3; `set_theta` holds c , s and the analytic G):

```

68 def set_theta(self, theta):
69     n = self.n
70     self.c = np.cos(theta)
71     self.s = np.sin(theta)
72     T = np.zeros((n, n))
73     T[self.ea, self.eb] = theta
74     T[self.eb, self.ea] = theta
75     CT = np.cos(T)
76     ST = np.sin(T)
77     np.fill_diagonal(CT, 0.0)
78     np.fill_diagonal(ST, 0.0)
79     Gcc = CT * CT
80     Gcs = CT * ST
81     Gss = ST * ST
82     np.fill_diagonal(Gcc, Gcc.sum(axis=1))
83     np.fill_diagonal(Gcs, Gcs.sum(axis=1))
84     np.fill_diagonal(Gss, Gss.sum(axis=1))

```

```

85         G = np.empty((2 * n, 2 * n))
86         G[:n, :n] = Gcc
87         G[:n, n:] = Gcs
88         G[n:, :n] = Gcs
89         G[n:, n:] = Gss
90         self.G = G

```

Parent iteration body (Eq. 4: lines 169–172; Eq. 5: lines 173–175; the criterion of Eq. 6: lines 176–181):

```

158 def make_parent(sys_lr, rng, iters=400, beta=0.5, tol=1e-8, restarts=3):
159     """自己無撞着円偏波固有モード親。小空間（JG）の固有対で反復。
160
161     収束判定（残差 < tol）付き。停滞時はランダム初期位相からリスタート。
162     """
163     best = (None, np.inf, None)
164     for _ in range(restarts):
165         theta = rng.uniform(0.0, 2.0 * np.pi, sys_lr.m)
166         v = None
167         for it in range(iters):
168             sys_lr.set_theta(theta)
169             ev, EV = np.linalg.eig(sys_lr.J @ sys_lr.G)
170             idx = int(np.argmin(ev.imag)) #  $\lambda = -i \sigma_{max}$ 
171             v = sys_lr.w(EV[:, idx].astype(complex))
172             v = v / np.linalg.norm(v)
173             theta_new = np.angle(v)
174             mix = (1.0 - beta) * np.exp(1j * theta) + beta * np.exp(1j * theta_new)
175             theta = np.angle(mix)
176             if it % 10 == 9:
177                 sys_lr.set_theta(np.angle(v))
178                 res_now = _eigenmode_residual(sys_lr, v)
179                 progress(f"親構成 iter={it+1} 残差={res_now:.2e}")
180                 if res_now < tol:
181                     break
182             sys_lr.set_theta(np.angle(v))
183             residual = _eigenmode_residual(sys_lr, v)
184             if residual < best[1]:
185                 best = (v, residual, sys_lr.sigma_spectrum())
186             if residual < tol:
187                 break
188     v, residual, sig = best
189     sys_lr.set_theta(np.angle(v))
190     return v, residual, sig

```

Residual (Eq. 6):

```

151 def _eigenmode_residual(sys_lr, v):

```

```

152     """カイラリティ非依存の固有モード残差:  $\mu = v^\dagger (iKv)$  に対する  $\| iKv - \mu v \|$ 。"""
153     kv = sys_lr.kmatvec(v)
154     mu = float(np.real(np.conj(v) @ (1j * kv)))
155     return float(np.linalg.norm(1j * kv - mu * v))

```

Stopping conditions and exceptional behavior (elements not formulated as equations): - The convergence check is performed only **every 10 iterations** (line 176, `it % 10 == 9`); the actual iteration count therefore ends at a multiple of 10. - If $r < \text{tol}$ is not reached within `iters=1200`, no exception is raised; the loop proceeds to a restart (line 165, consuming new initial phases from the rng), up to `restarts=3`. The solution with the best residual is adopted (lines 163, 184–185). **A restart changes the number of rng draws**, so reproduction requires the same `tol`. (In this run, every N converged within the first restart with no extra draws; evidence: the $N=40$ products are bit-identical to the canonical generated under the same conditions.) - The **progress** line (line 179) is `stderr` output only and does not affect the computation.

■4.3.3 Loop (2): Construction of the Zero-Closure Kernel Seed (Eq. 7)

```

193 def zero_closure_kernel_seed(sys_lr, rng):
194     """span(W) の直交補内の実正規直交対 (u,w) から  $g=(u+iw)/\sqrt{2}$ 。  $g^T g = 0$  厳密。"""
195     m = sys_lr.m
196     def project_out(g):
197         q = np.linalg.lstsq(sys_lr.G, sys_lr.wt(g), rcond=None)[0]
198         return g - sys_lr.w(q)
199     u = project_out(rng.normal(size=m))
200     u = u / np.linalg.norm(u)
201     w = project_out(rng.normal(size=m))
202     w = w - (w @ u) * u
203     w = w / np.linalg.norm(w)
204     return (u + 1j * w) / math.sqrt(2.0)

```

- The projection ($P^\perp$ of Eq. 7) solves the normal equations $Gq = W^T g$ by `lstsq` (`rcond=None`; line 197). Even for near-singular G , `lstsq` returns a least-squares solution, so no exception arises.
- The G at this point is that of $\theta = \arg v$ set at the end of `make_parent` (line 189); i.e., the seed is taken from the orthogonal complement of the range of the parent's generator.

■4.3.4 Finalization: Z0 Construction, Gate, Saving, Ledger `make_static_parents_N3_N40_v1.py` (Eq. 8: lines 52–53):

```

50     v, residual, sig = make_parent(sys_lr, rng, iters=ITERS, tol=TOL)
51     g = zero_closure_kernel_seed(sys_lr, rng)
52     Z = v + DELTA * g
53     Z = Z / np.linalg.norm(Z)
54     converged = bool(residual < 1e-8)
55     status = "ok" if converged else "NOT_CONVERGED"
56
57     if N == 40:

```

```

58         ref = np.load(REF40)
59         same = all(np.array_equal(x, ref[k]) for x, k in ((v, 'v'), (g, 'g'), (Z, 'Z0')))
60         print(f"GATE N=40 v/g/Z0 bit-identical to canonical static parent: {same}")
61         if not same:
62             gate_ok = False
63             status = "GATE_FAIL"
64
65     if os.path.exists(out_path):
66         prev = np.load(out_path)
67         same_prev = all(np.array_equal(x, prev[k]) for x, k in ((v, 'v'), (g, 'g'), (Z,
68         print(f"N={N}: 既存ファイルあり（上書きせず検証のみ）一致={same_prev} residual={residual}")
69         if not same_prev:
70             gate_ok = False
71             status = "EXISTING_MISMATCH"
72     else:
73         np.savez_compressed(out_path, v=v, g=g, Z0=Z,
74                             sigma=sig, residual=np.float64(residual),
75                             n=np.int64(N), seed=np.int64(SEED),
76                             delta=np.float64(DELTA), tol=np.float64(TOL),
77                             iters=np.int64(ITERS))
81     with open(os.path.join(PARENT_DIR, "parents_summary.csv"), "w", newline="") as fh:
82         w = csv.writer(fh)
83         w.writerow(["N", "M", "parent_residual", "rank_planes", "status"])
84         w.writerows(rows)
85     n_bad = sum(1 for r in rows if r[4] != "ok")
86     print(f"summary: {len(rows)} parents, {n_bad} non-ok")
87     if not gate_ok:
88         print("GATE FAIL")
89         sys.exit(1)
90     print("STATIC PARENTS DONE")

```

Exceptional behavior: - Existing files are **never overwritten**. If a file exists, only bit-identity verification against the regenerated values is performed (lines 65–71); a mismatch is flagged EXISTING_MISMATCH and fails the gate. - Non-convergence (residual $\geq 1e-8$) is not an exception; it is explicitly recorded in the ledger as NOT_CONVERGED (lines 54–55). - Any gate failure yields exit code 1 (lines 87–89).

5. Execution Results

5.1 Reproduction Commands

```

cd N3_N40_stage123_sweep_20260905
python3 make_static_parents_N3_N40_v1.py      # standalone
# or as the first step of ./run_all.sh

```

5.2 Execution Environment

- Python 3.9.6 (`.venv/bin/python3`)
- numpy 2.0.2 (BLAS/LAPACK: macOS Accelerate)
- macOS 26.3.1 (arm64, Darwin 25.x)
- RNG: numpy `default_rng` (PCG64)

5.3 Execution Time

Roughly 30 seconds to 1 minute for all 38 parents (including generation, saving, verification, and ledger output). The dominant cost per parent is the eigendecomposition of JG ($2N \times 2N$) times the iteration count; ~ 0.15 s per parent construction at $N=40$.

5.4 Verification Gates

Gate	Pass condition	Measured	Verdict
G1: N=40 lineage	generated v, g, Z0 equal (via <code>np.array_equal</code>) to the canonical static parent (20260904; itself verified bit-identical to the July canonical run)	equal	PASS
G1': file identity	(observation) SHA256 of the N=40 npz equals that of the canonical static-parent file	both <code>eadc87ee...</code> , identical	PASS
G2: convergence	residual $< 1e-8$ for all N (status=ok)	38/38 ok, residual $\in [3.54e-14, 9.42e-13]$	PASS
G3: execution	exit code 0 and STATIC PARENTS DONE printed	confirmed	PASS

5.5 Data

Item	Content
Folder	<code>N3_N40_stage123_sweep_20260905/parents/</code>
Static-parent npz	<code>parent_static_N00003..N00040_makeparent_20260905.npz</code> (38 files)
Sizes	~ 2.0 KB ($N=3$; 1,980 bytes) to ~ 37 KB ($N=40$; 37,396 bytes), total ~ 636 KB
Ledger	<code>parents_summary.csv</code> (38 rows: N, M, <code>parent_residual</code> , <code>rank_planes</code> , status)
SHA256	canonical values for all files in the bundled <code>SHA256SUMS.txt</code> . Representative values:


```

c3230103e82976decde7bbe6fe5df545d12804127cf274f0d385e044ce544ca2  make_static_parents_N3_N40_v1.py
ba0fc19b03caf06d16e97e2cd5da499ed2ed8e95288f6dbf2062bedcaf11176d  run_n_scaling_lowrank_v1.py
eadc87ee0276554c7ab02e571e05200f0b719c1250b82607c7546b30a4d6f232  parents/parent_static_N00040_makep
f384d9e90cc0a3a87a82a9a6928c9996d51bd0f83862c79f2cfe8a5ce182bbca  parents/parents_summary.csv

```

5.6 Figures

No figures are generated in this chapter (the step-0 grid figure of Chapter 3, `fig_complex_plane_step0_N3_N40_stage123`, visualizes this chapter’s products).

6. Execution Analysis (objective report and observations only)

1. All 38 parents converged within the first restart; the eigenmode residuals fell in the range 3.54×10^{-14} to 9.42×10^{-13} (all at or below the order of $\text{tol}=1\text{e-}12$). Zero cases of NOT_CONVERGED or GATE_FAIL.
2. The $N=40$ products are bit-identical, at the array level, to the canonical static parent generated independently (on 2026-09-04, in a different folder), and the SHA256 of the npz files themselves also coincide. This is direct evidence that the generation is fully deterministic under the same seed, engine, and environment.
3. The number of nonzero planes of the parent’s σ spectrum (`rank_planes`) was observed as $N=3$: 2, $N=4$: 2, $N=5$: 4, $N=6$: 6, and N for $N \geq 7$ (see the ledger `parents_summary.csv`).
4. File sizes grow roughly in proportion to $M = N(N - 1)/2$.
5. The rng consumption is the minimal “ m uniforms + $2m$ normals” (when no restart occurs); every run completed with this minimal consumption (the bit-identity of item 2 is corroborating evidence).

(End of Chapter 1. Chapter 2, “The Sweep — Stage 1+2+3 Dynamics”, follows.)